

Tecnológico de Estudios Superiores de Villa Guerrero

Ingeniería en sistemas computacionales

Unidad de Aprendizaje:

Inteligencia Artificial

Nombre de la práctica:

Manual de practica

Presenta:

García Mendoza Jazmín

Profesor:

L.I Carbajal Bravo Emmanuel

Semestre: Octavo

Grupo:3801

LA FINCA, VILLA GUERRERO, ESTADO DE MÉXICO.

ENERO DE 2026





Contenido

Introducción	3
Practica 1 Localization.....	4
Practica 2 Bayesianas.....	6
Practica 3 Agente De Inteligencia Artificial.....	8
Conclusión	10
Referencia	11

Tabla de ilustraciones

Ilustración 1 Función Actualizar.....	4
Ilustración 2 Rutas al TEC	5
Ilustración 3 Se crea red bayesiana.....	6
Ilustración 4 Mostrar grafo	7
Ilustración 5 Red bayesiana	7
Ilustración 6 Herramientas del agente.....	8
Ilustración 7 Da resultado	9



Introducción

La Inteligencia Artificial (IA) ha experimentado un crecimiento significativo en los últimos años debido al desarrollo de modelos de lenguaje capaces de comprender y generar texto de manera natural. Sin embargo, estos modelos presentan limitaciones cuando requieren realizar cálculos, consultar información actualizada o interactuar con servicios externos. Para superar estas restricciones, se han desarrollado agentes inteligentes que combinan las capacidades de razonamiento de los modelos de lenguaje con herramientas especializadas que permiten ejecutar acciones concretas y proporcionar respuestas más precisas.

En la presente práctica se implementó un agente de Inteligencia Artificial utilizando el lenguaje de programación Python, integrando diversas herramientas para efectuar operaciones matemáticas, consultar información del clima, obtener la fecha y hora actual, y realizar búsquedas de información. Asimismo, se empleó el patrón ReAct (Reasoning and Acting), el cual permite que el agente analice una solicitud, seleccione la herramienta adecuada, procese el resultado obtenido y genere una respuesta final basada en evidencia. Con ello, se demuestra cómo la integración de modelos de lenguaje con herramientas externas amplía las capacidades de los sistemas inteligentes y mejora la interacción entre el usuario y la aplicación.

Practica 1 Localization

En esta práctica se desarrolló una aplicación web sencilla utilizando **HTML** y **JavaScript**, cuyo propósito es permitir al usuario seleccionar una ruta de traslado mediante una lista desplegable y visualizar información relacionada con el recorrido. La aplicación almacena los datos de cada ruta en un arreglo de objetos que contiene la distancia, el tiempo estimado, el costo aproximado y el enlace correspondiente para visualizar el recorrido en Google Maps.

Ilustración 1 Función Actualizar

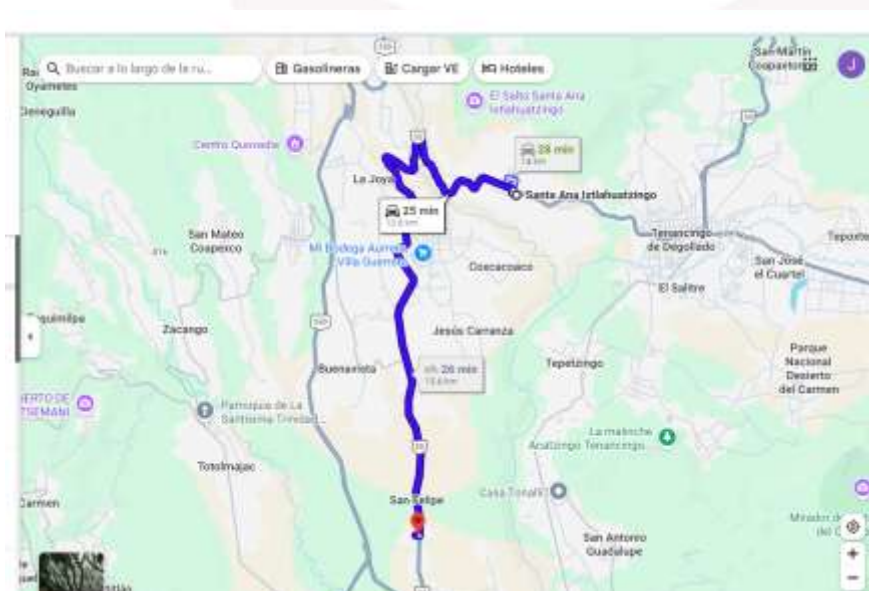
```
function actualizar(){  
  
    let indice =  
    document.getElementById("ruta").value;  
  
    document.getElementById("distancia").textContent =  
    rutas[indice].distancia;  
  
    document.getElementById("tiempo").textContent =  
    rutas[indice].tiempo;  
  
    document.getElementById("costo").textContent =  
    rutas[indice].costo;  
  
}
```

Al seleccionar una opción del menú, JavaScript actualiza automáticamente la información mostrada en la página mediante la manipulación del **Document Object Model (DOM)**, proporcionando una interacción dinámica sin necesidad de recargar el sitio web. Además, se implementó un botón que abre en una nueva pestaña el recorrido seleccionado utilizando la función `window.open()`, permitiendo consultar la ruta directamente en Google Maps.

La práctica demuestra el uso de estructuras de datos, funciones, eventos y manipulación de elementos HTML para desarrollar interfaces web interactivas. Asimismo, permite comprender la integración entre aplicaciones desarrolladas con JavaScript y servicios externos de navegación mediante enlaces dinámicos.

Resultados

Ilustración 2 Rutas al TEC



Practica 2 Bayesianas

En esta práctica se desarrolló una aplicación en **Python** utilizando la biblioteca **pgmpy** para construir una **Red Bayesiana** orientada al análisis probabilístico del uso de la Inteligencia Artificial (IA) en diferentes carreras profesionales. El programa permite al usuario seleccionar una carrera de una lista disponible y, a partir de esta elección, genera un modelo con variables específicas relacionadas con el área de conocimiento correspondiente.

Ilustración 3 Se crea red bayesiana

```
# =====  
# PROBABILIDADES  
# =====  
  
modelo = DiscreteBayesianNetwork([  
    ("UsoIA", nodo2),  
    (nodo2, nodo3)  
)  
  
cpd_ia = TabularCPD(  
    variable="UsoIA",  
    variable_card=2,  
    values=[  
        [0.30],  
        [0.70]  
    ]  
)
```

La red está compuesta por tres nodos: **UsoIA**, una variable intermedia que cambia de acuerdo con la carrera seleccionada y un nodo final que representa el resultado esperado, como la aprobación de un curso, la eficiencia, la productividad o la satisfacción del usuario, según el contexto. Posteriormente, se definen las distribuciones de probabilidad condicional para cada variable y se valida que la estructura del modelo sea consistente antes de realizar el proceso de inferencia.

Ilustración 4 Mostrar grafo

```
G = nx.DiGraph()

G.add_edge("UsoIA", nodo2)
G.add_edge(nodo2, nodo3)

plt.figure(figsize=(8,4))

pos = nx.spring_layout(G)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=3500,
    font_size=10
)
```

Como parte del desarrollo, el programa representa gráficamente la red mediante las bibliotecas **NetworkX** y **Matplotlib**, permitiendo visualizar las relaciones de dependencia entre las variables. Finalmente, se aplica el algoritmo de **Eliminación de Variables (Variable Elimination)** para calcular la probabilidad de que un estudiante haya utilizado Inteligencia Artificial con base en la evidencia observada del resultado final, mostrando la probabilidad obtenida y su interpretación.

Resultados

Ilustración 5 Red bayesiana

```
3 - Ingeniería Industrial
4 - Ingeniería en Industrias Alimentarias
5 - Licenciatura en Administración
6 - Licenciatura en Turismo
7 - Licenciatura en Arquitectura
8 - Ingeniería Mecatrónica

Seleccione una carrera: |
```

Practica 3 Agente De Inteligencia Artificial

En esta práctica se desarrolló un **agente de Inteligencia Artificial en Python** capaz de resolver diferentes tipos de consultas mediante el uso de herramientas externas. El sistema implementa el patrón **ReAct (Reasoning and Acting)**, permitiendo que el modelo analice la solicitud del usuario, determine qué herramienta necesita utilizar, procese la información obtenida y genere una respuesta final con base en los resultados.

Ilustración 6 Herramientas del agente

```
# 1. DEFINIMOS LAS HERRAMIENTAS QUE EL AGENTE PUEDE USAR
def calcular_al_cuadrado(numero):
    """Multiplica un número por sí mismo."""
    print(f" -> [Herramienta]: Calculando el cuadrado de {numero}...")
    return int(numero) ** 2

def calcular_raiz_cuadrada(numero):
    """Calcula la raíz cuadrada de un número."""
    print(f" -> [Herramienta]: Calculando raíz cuadrada de {numero}...")
    if int(numero) < 0:
        return "Error: No se puede calcular raíz de número negativo"
    return math.sqrt(int(numero))

def obtener_clima(ciudad):
    """Obtiene el clima actual de una ciudad usando wttr.in"""
    print(f" -> [Herramienta]: Consultando clima en {ciudad}...")
    try:
        r = requests.get(f"https://wttr.in/{ciudad}?format=3", timeout=5)
        return r.text.strip()
    except:
        return "Error: No pude obtener el clima"
```

El agente incorpora diversas funciones para realizar operaciones específicas, entre ellas el cálculo del cuadrado y la raíz cuadrada de un número, la consulta del clima de una ciudad mediante un servicio web, la obtención de la hora actual y la simulación de búsquedas de información en Internet. Todas estas funciones son registradas en un catálogo de herramientas para que el modelo pueda seleccionarlas automáticamente según la intención de la consulta realizada por el usuario.

Para controlar la interacción con el modelo de lenguaje, se definió un **System Prompt** que establece las reglas de funcionamiento del agente y obliga a que todas las respuestas sean generadas en formato **JSON**, indicando el razonamiento, la acción a ejecutar, el argumento correspondiente y la respuesta final. Posteriormente, el programa interpreta dicho formato, ejecuta la herramienta solicitada y devuelve el resultado al modelo para continuar el proceso de razonamiento hasta obtener una solución definitiva.

Ilustración 7 Da resultado

```
    resultado = HERRAMIENTAS[nombre_herramienta](argumento)

    # Devolver resultado al contexto
    mensajes.append(("role": "assistant", "content": contenido_respuesta))
    mensajes.append(("role": "user", "content": f"Observación: {resultado}"))
else:
    print(f"Error: Herramienta '{nombre_herramienta}' no existe.")
    mensajes.append(("role": "user", "content": f"Observación: Error, la herramienta no existe"))

# Caso B: Respuesta final
else:
    print(f"\nAgente: {datos['respuesta_final']}")
    return datos['respuesta_final']

print("\nAgente: Se alcanzó el límite de pasos sin llegar a una respuesta final.")
return None
```

Finalmente, se implementó un ciclo de ejecución que permite realizar múltiples pasos de inferencia, gestionar errores durante la comunicación con el modelo de lenguaje y validar las respuestas generadas. Como prueba de funcionamiento, el agente resuelve consultas matemáticas, obtiene información del clima y la hora actual, además de realizar búsquedas simuladas, demostrando la integración entre modelos de lenguaje y herramientas programadas en Python.

Conclusión

La práctica permitió comprender el funcionamiento de un agente de Inteligencia Artificial basado en el patrón ReAct, integrando un modelo de lenguaje con herramientas desarrolladas en Python para resolver diferentes tipos de consultas. La implementación demostró que la combinación del razonamiento automático con funciones especializadas incrementa la capacidad del sistema para proporcionar respuestas útiles, precisas y adaptadas a las necesidades del usuario.

Además, se comprobó la importancia de definir un flujo de trabajo estructurado mediante el uso de un *System Prompt*, el intercambio de información en formato JSON y la ejecución controlada de herramientas externas. Estos elementos favorecen la construcción de agentes más robustos y flexibles, capaces de interactuar con servicios web, realizar cálculos y procesar información en tiempo real. En conjunto, la práctica permitió fortalecer los conocimientos sobre Inteligencia Artificial, programación en Python y el desarrollo de agentes inteligentes orientados a la automatización de tareas.

Referencia

Carnegie Mellon University. (s. f.). *Artificial Intelligence Repository*.

<https://www.cs.cmu.edu/afs/cs/project/ai-repository/ai/html/ai.html>

Harvard University. (s. f.). *CS50's Introduction to Artificial Intelligence with Python*.

<https://cs50.harvard.edu/ai/>

Massachusetts Institute of Technology. (s. f.). *MIT OpenCourseWare: Artificial Intelligence*.

<https://ocw.mit.edu/courses/6-034-artificial-intelligence-fall-2010/>

National Institute of Standards and Technology. (2023). *Artificial Intelligence*.

<https://www.nist.gov/artificial-intelligence>

Stanford University. (2025). *Artificial Intelligence Index Report 2025*. <https://hai.stanford.edu/ai-index>

University of California, Berkeley. (s. f.). *CS 188: Introduction to Artificial Intelligence*.

<https://inst.eecs.berkeley.edu/~cs188/>

U.S. Department of Energy. (2024). *Artificial Intelligence*. <https://www.energy.gov/ai>

U.S. National Science Foundation. (2024). *Artificial Intelligence*. <https://new.nsf.gov/focus-areas/artificial-intelligence>